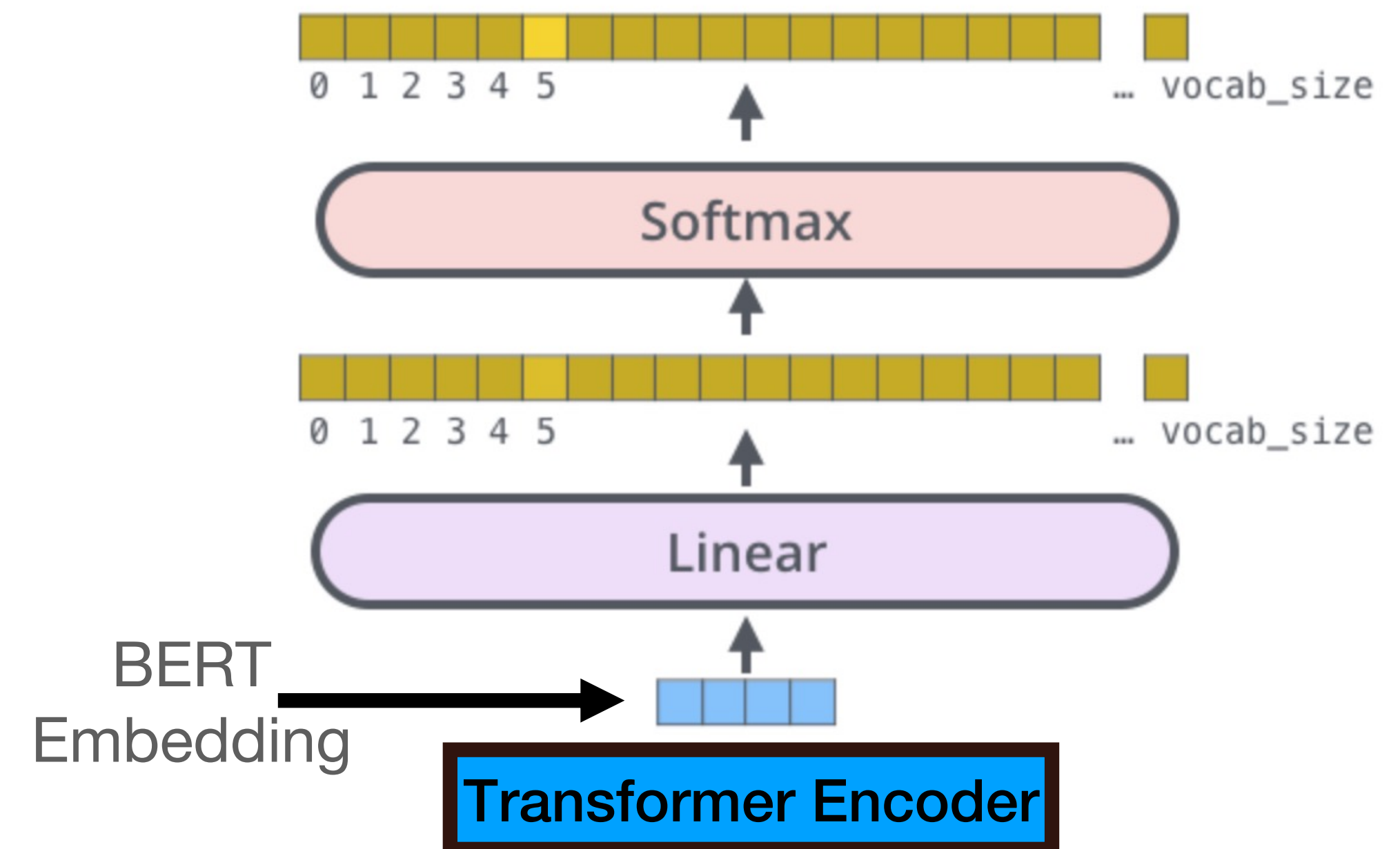


The BERT Model

- The first widely popular model to use pre-training
- The BERT model is a standard model for obtaining contextualized word embeddings
- The model is built like a *Transformer encoder*, except that instead of generating a list of vector embeddings, it feeds these vectors to a linear layer, followed by a soft-max
- BERT provides embeddings for each token in the input sequence – the computed vector that enters the linear layer



BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

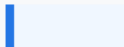
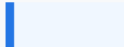
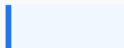
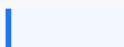
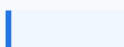
<https://arxiv.org/abs/1810.04805>

Masked Language Models

- BERT is trained as a **masked language model** (MLM)
- MLMs are statistical models that, given a sentence where part of the tokens are replaced with masks, predict the identity of the masked tokens

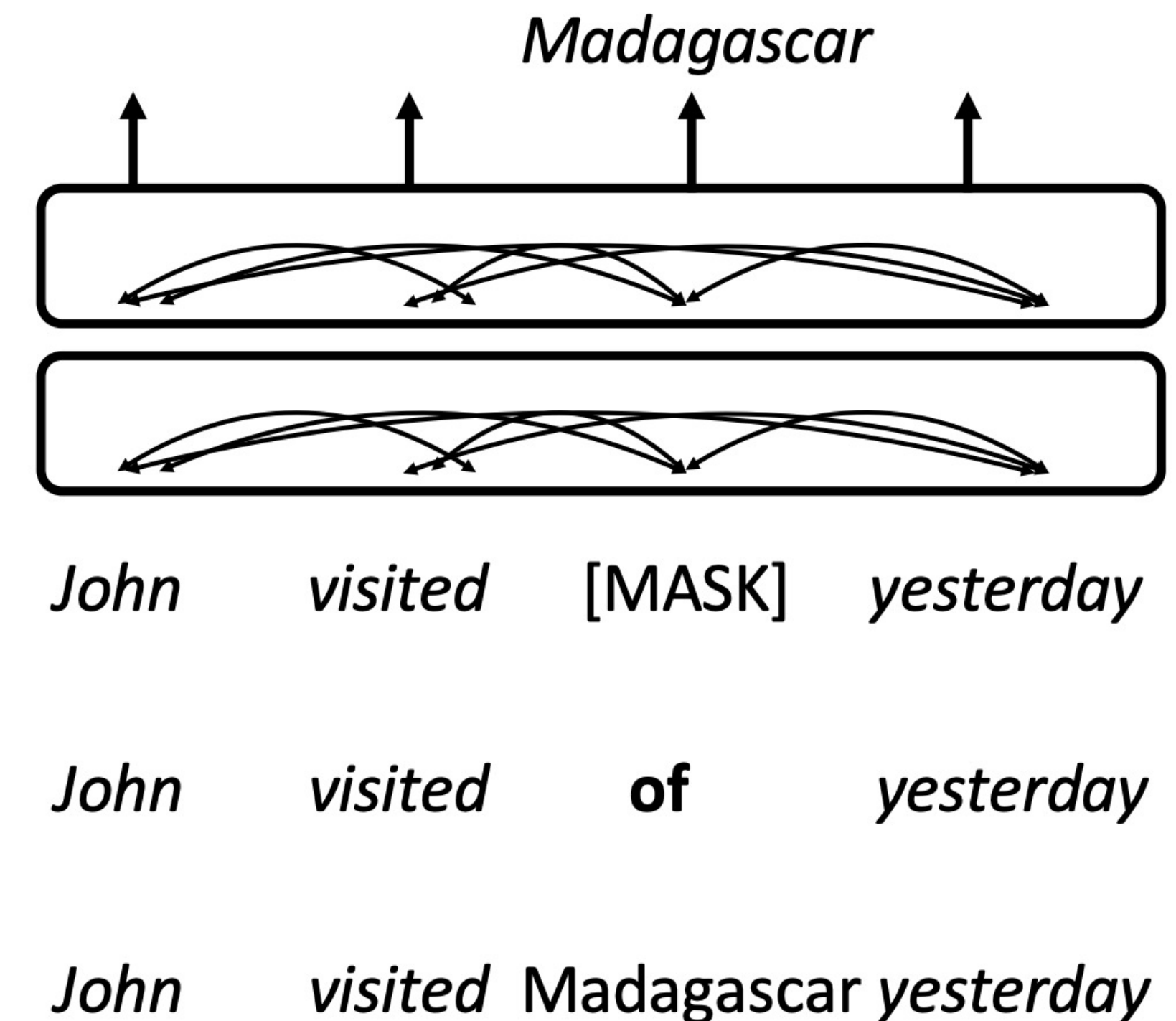
My dog is [MASK] and likes to [MASK] the entire [MASK].

Mask 1

Prediction	Score
My dog is hungry and likes to [MASK2] the entire [MASK3] .	 5.6%
My dog is friendly and likes to [MASK2] the entire [MASK3] .	 4.8%
My dog is cute and likes to [MASK2] the entire [MASK3] .	 3.7%
My dog is nice and likes to [MASK2] the entire [MASK3] .	 2.9%
My dog is smart and likes to [MASK2] the entire [MASK3] .	 2.4%

BERT's Training

- BERT's training is carried out by segmenting texts into windows in the size of the model's input window, and selecting 15% of the tokens (denote the set with S)
- The tokens in S are altered such that:
 - 80% of them are replaced with the special token <MASK>
 - 10% of them are replaced with a random token
 - 10% of them remain the same
- Denote the modified input sequence with $\mathbf{x}'$

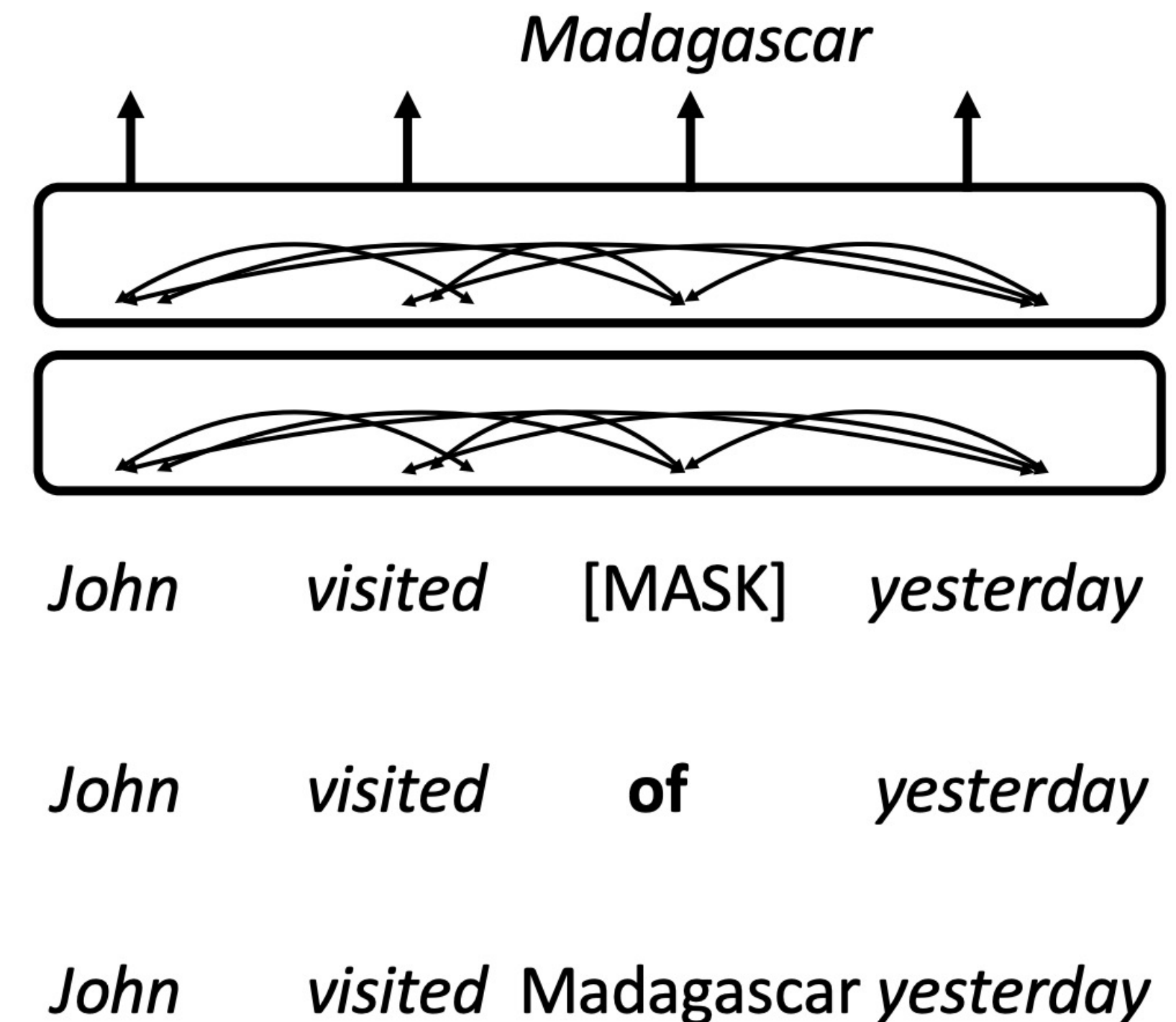


BERT's Training

- The loss function is then the cross entropy loss over the tokens in S:

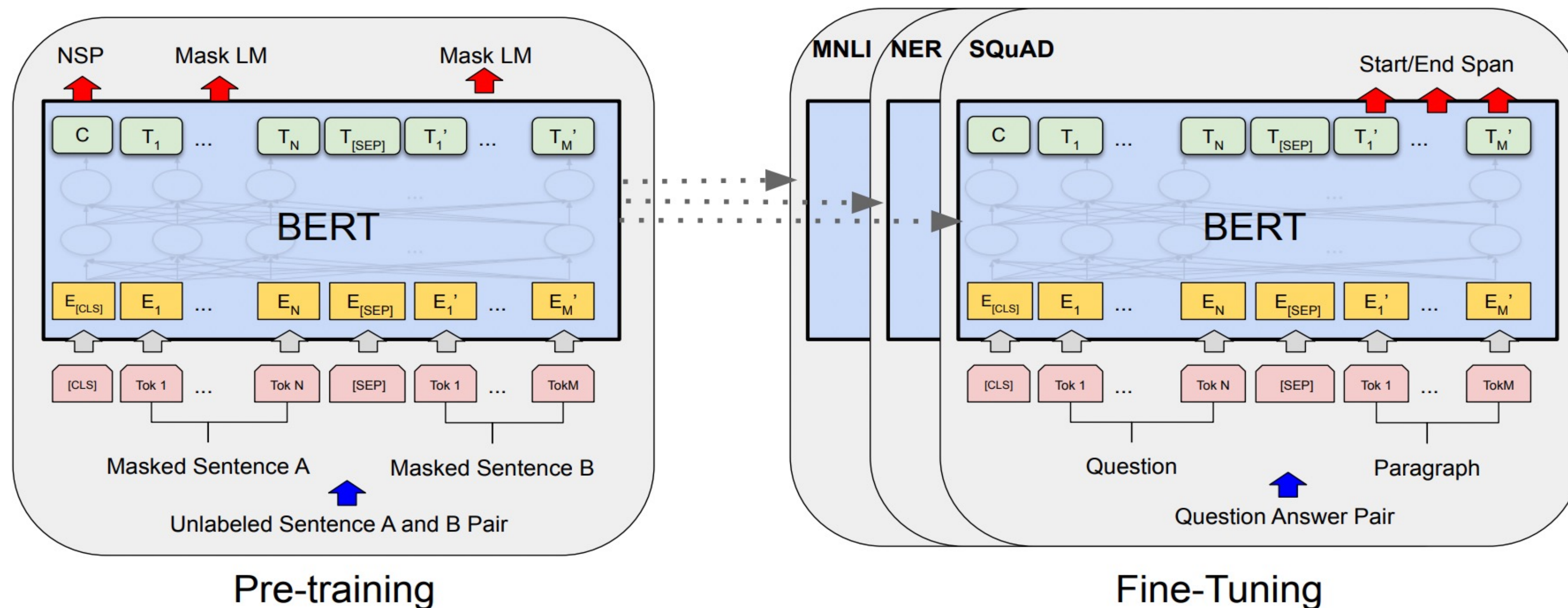
$$L(\theta) = -\frac{1}{|S|} \sum_{i \in S} \log(p_{\theta}(x_i|x'))$$

- During inference, no masking is employed



Fine Tuning BERT

- The standard paradigm for using BERT in NLP includes two phases:
 - Pre-training
 - Fine-tuning



Examples for fine-tuning BERT

- POS tagging:
 - Often viewed as an independent prediction task (not structured prediction)
 - A classification head for each word
 - Its input is the BERT embedding and its output is a soft-max in the dimension of the number of POS tags.
 - As usual, the j -th coordinate of the soft-max is interpreted as $P(y_j/x_1, \dots, x_n)$
 - Training set:
 - Each sample consists of the sentence $(x_1, \dots, x_n)$, index i , and gold POS tag for i -th token y_i^*
 - Cross entropy loss over $P(y_i^*/x_1, \dots, x_n)$

Examples for fine-tuning BERT

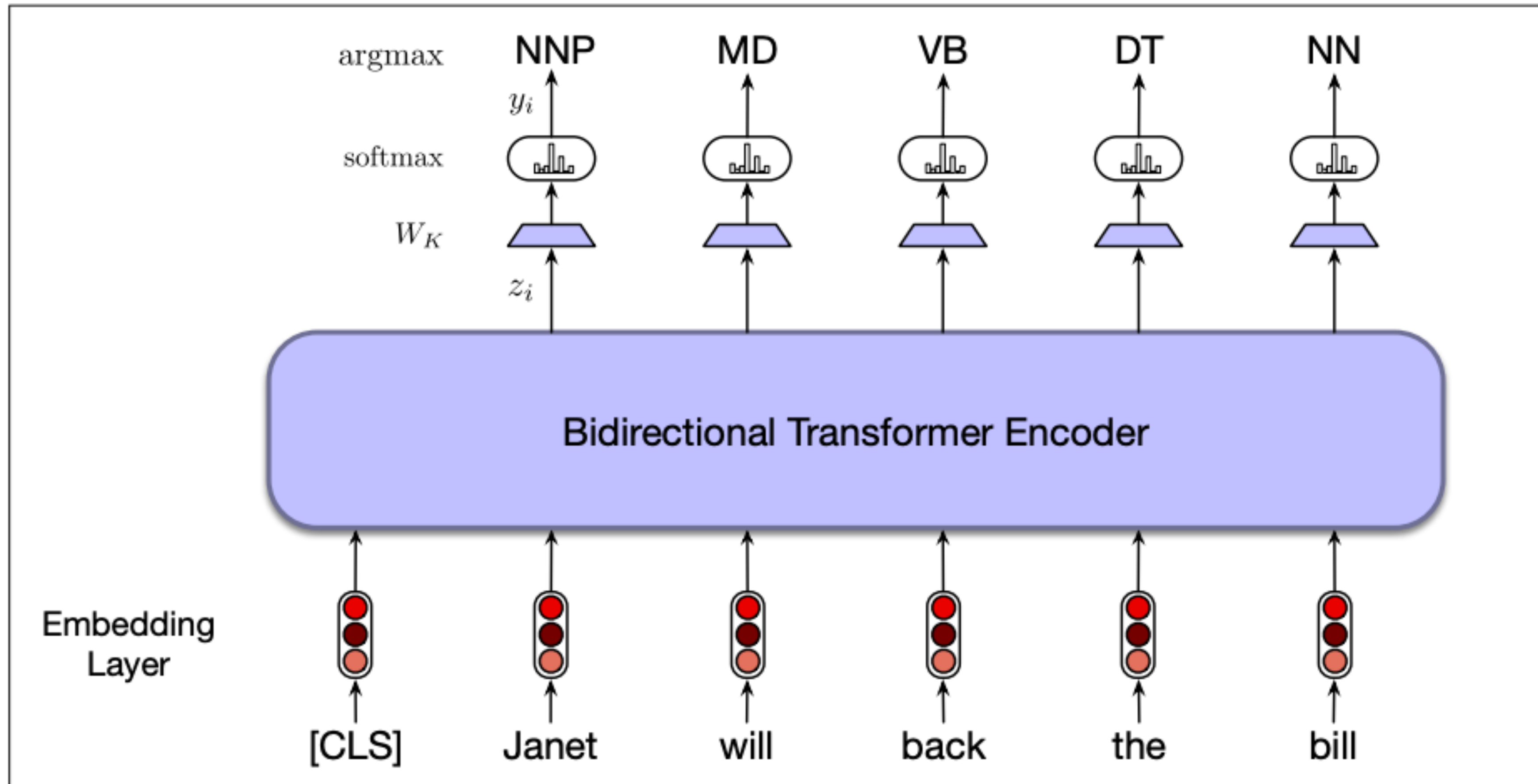


Figure 11.9 Sequence labeling for part-of-speech tagging with a bidirectional transformer encoder. The output vector for each input token is passed to a simple k-way classifier.

Next Sentence Prediction

- BERT is in fact pretrained not only as a masked language, but also as a model for Next Sentence Prediction (NSP)
- NSP is the task that requires, given two sentences, to determine whether the two sentences form a sensible consecutive pair or is just two random sentences
 - NSP training data: positive - pairs of adjacent sentences, negative - pairs of random sentences

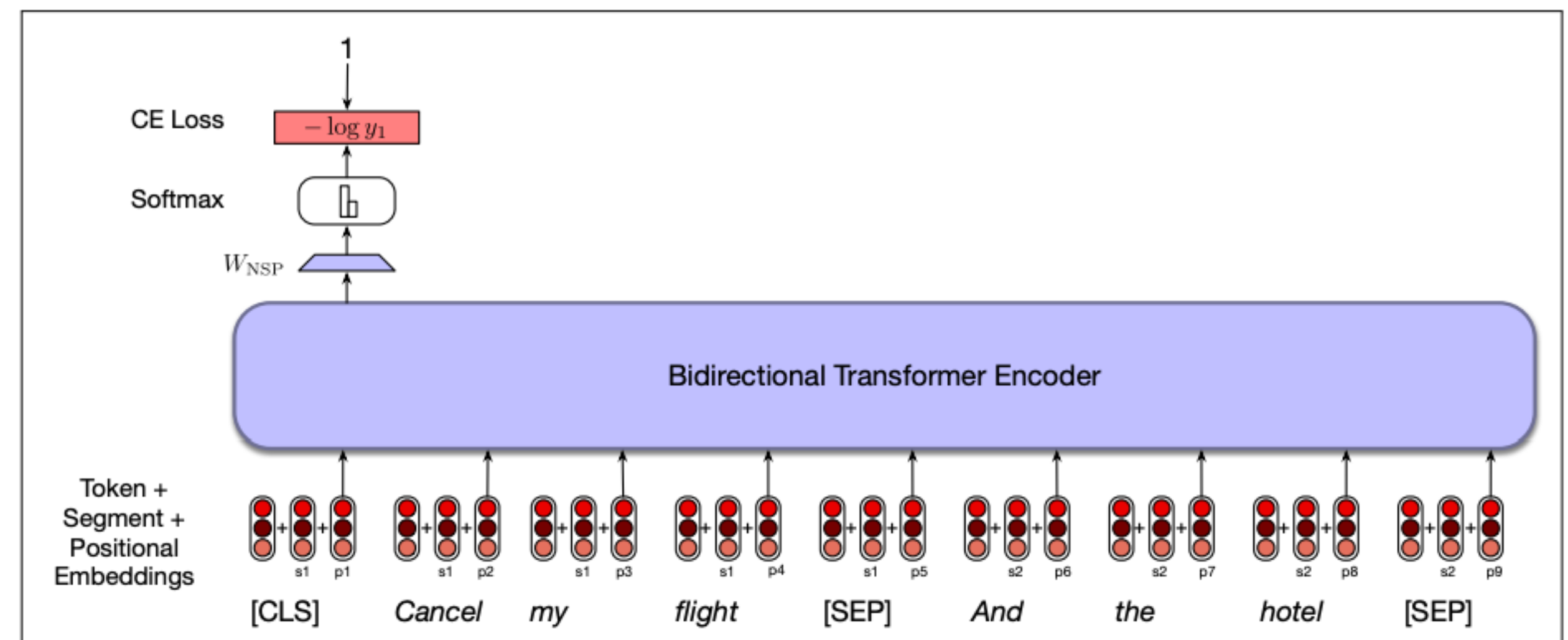


Figure 11.7 An example of the NSP loss calculation.

Next Sentence Prediction

- Each training sample is prepended with a special [CLS] token. A special [SEP] token is added between the sentences
- The loss function is over the [CLS] where the labels are 1/0
- [CLS] is often used for fine-tuning on sentence classification tasks

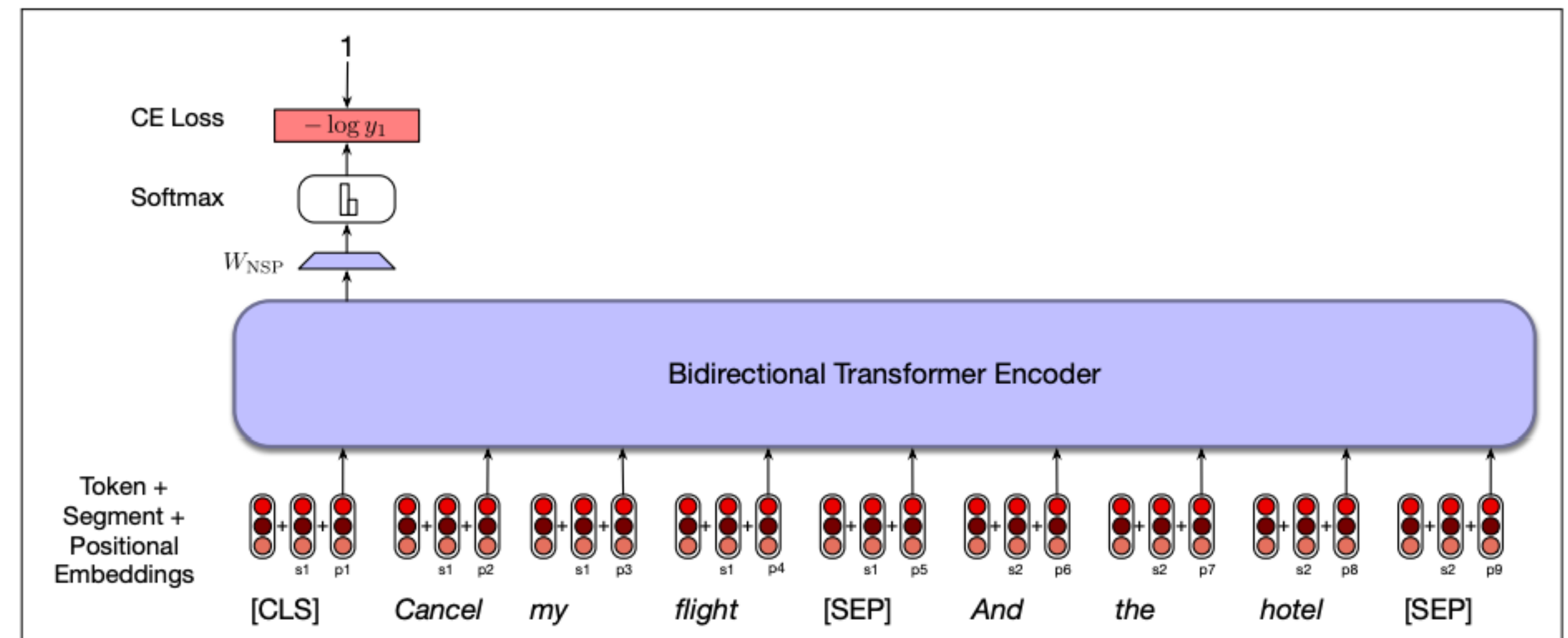


Figure 11.7 An example of the NSP loss calculation.

Examples for fine-tuning BERT

- Sentiment analysis:
 - Viewed as a sentence classification task
 - A classification head is placed over a special token ([CLS])
 - As before, usually a linear layer followed by a soft-max
 - The soft-max is interpreted as $P(class/x_1, \dots, x_n)$
 - *class* takes values in the set of potential sentiment classes
- Training set:
 - Each sample consists of the sentence $(x_1, \dots, x_n)$, gold POS tag for the sentence y^*
 - The loss function is the expectation over $\log P(y^* / x_1, \dots, x_n)$

Results of Fine-tuning BERT

- BERT achieved very impressive results across a range of token and sentence classification tasks, sequence prediction, as well as other baselines
- A very broad experimental setup in terms of task types and baselines
- It was widely adopted, and is still often used (about 5 years after its inception)
- For detailed results, see the original BERT paper